

Desafio Técnico: Simulador de Financiamentos - JAVA

1. O Cenário

O candidato foi encarregado de construir o protótipo backend de uma API de simulação de financiamentos e investimentos. O objetivo principal é receber os parâmetros básicos de uma operação de crédito, calcular os juros compostos devidos ao longo do tempo, gerar uma memória de cálculo detalhada (evolução mês a mês) e persistir esse histórico na base de dados para consultas futuras.

2. Requisitos Técnicos e Stack

O candidato deve escolher e implementar a solução em apenas uma das seguintes stacks:

- **Linguagem:** Java 25
- **Framework:** Quarkus
- **Base de Dados:** H2 Database (em modo embutido: ficheiro ou in-memory)

Atenção: O uso de Docker ou Docker Compose está expressamente proibido neste desafio. A aplicação e os seus testes devem ser executados de forma 100% nativa utilizando apenas as ferramentas da SDK instaladas localmente na máquina.

3. Requisitos Funcionais

3.1. Simular e Persistir Cálculo de Juros

- Input (Payload de Entrada):
 - valorInicial (Principal - ex: 1000.00)
 - taxaJurosMensal (Em formato percentual - ex: 1.5 para 1,5% ao mês)
 - prazoMeses (Inteiro representando o tempo de contrato - ex: 12)
- Processamento:
 - Aplicar a fórmula de Juros Compostos para determinar a evolução do saldo devedor.
 - Gerar uma Memória de Cálculo detalhada passo a passo, registrando o saldo no início de cada período, a parcela correspondente de juros incidentes e o saldo final desse mês.
- Retorno & Persistência:

O cálculo deve ser guardado na base de dados e retornar:

 - ID único da simulação.
 - Valor Total Final.
 - Valor Total de Juros.
 - Lista da Memória de Cálculo (Mês, Saldo Inicial, Juro, Saldo Final).

3.2. Consultar Simulação Existente

Input: ID da simulação via parâmetro de rota.

Retorno: Objeto completo da simulação contendo todos os parâmetros de entrada, totais calculados e a memória de cálculo completa associada.

4. Requisitos Não Funcionais e Engenharia

Cobertura de Testes Automatizados (Mínimo 80%): Este é um critério estrito e eliminatório. Deve possuir testes de unidade e integração. Utilizar Jacoco (Java) para o relatório.

Design de API (Spec-Driven Development): Os contratos da API precisam de ser desenhados com extremo rigor técnico. O projeto deve expor a especificação OpenAPI / Swagger de forma automática.

Precisão Financeira: A manipulação de valores monetários deve prevenir erros de arredondamento. Espera-se o uso de tipos de alta precisão como BigDecimal (Java)

5. Matriz de Critérios de Avaliação

Pilar de Avaliação	Esperado	Falha Crítica
1. Rigor nos Testes (Eliminatório)	Atingiu 80%+ de cobertura reportada. Testou cenários de erro e borda.	Cobertura abaixo de 80%. Testes vazios ou sem asserções.
2. Precisão Financeira	Utilizou BigDecimal/decimal. Cálculo de juros e arredondamento corretos.	Utilizou double ou float. Perda de precisão.
3. Spec-Driven	Respeito ao contrato OpenAPI. Respostas HTTP corretas (400, 404, 201).	Alterou payload de saída. Retorna HTTP 500 para falhas de validação.
4. Persistência Embutida	Executa nativamente e cria as tabelas do H2 de forma automática.	Exigiu contentores Docker ou execução de scripts SQL manuais.
5. Clean Code	Camadas bem definidas (Resource -> Service -> Repository).	Regras de negócio acopladas no Controller. Nomenclatura confusa.

6. Instruções de Entrega

O código-fonte deve ser fornecido por formato compactado (.ZIP), anexado na sua produção temática, devendo conter obrigatoriamente um ficheiro README.md com as instruções claras de compilação e o comando exato a ser executado no terminal para correr a suíte de testes e validar a cobertura de 80%.